

ESTRUCTURAS DE DATOS - 2026-2

Profesor: David Herrera

Monitora: Ángela Camila Siabato Londoño

TAREA INDIVIDUAL.

Implementación y Análisis de Complejidad de Listas , Pilas y Colas en Java

OBJETIVO

Implementar las estructuras de datos **Stack (pila)** y **Queue (cola)** en **Java**, utilizando tanto **arreglos dinámicos** como **listas enlazadas**, realizando un análisis de complejidad de los métodos asociados.

DESCRIPCIÓN DE LA TAREA

1. Implementación de la Estructura List:

- a. Implementar la estructura de datos **List** basada en listas enlazadas (**sencillamente enlazadas, sin y con cola, y doblemente enlazadas sin y con cola**) realizando un análisis de complejidad de los métodos asociados
- b. Los métodos que deben implementar y analizar son:
 - **PushFront**: Inserción de un elemento al inicio de la lista.
 - **PushBack**: Inserción de un elemento al final de la lista.
 - **PopFront**: Eliminación del primer elemento de la lista.
 - **PopBack**: Eliminación del último elemento de la lista.
 - **Find**: Retornando la referencia del elemento en la lista.
 - **Erase**: Eliminación del elemento indicado de la lista.
 - **AddBefore**: Inserción de un elemento antes del elemento indicado en **la lista**.
 - **AddAfter**: Inserción de un elemento después del elemento indicado en la lista.
 - **Otros métodos relevantes que consideren necesarios (p. ej., Empty, TopBack, etc.)**

2. Implementación en Java Pilas y Colas

a. Implementar las interfaces `MyStack<T>` y `MyQueue<T>` con los métodos fundamentales.

- Implementación de ambas interfaces con **arreglo dinámico** (Un ejemplo nativo en Java sería `ArrayList`). Recuerden que el tamaño se debe aumentar a medida que el tamaño del contenedor vaya creciendo. Escoger alguna forma de incrementar el tamaño de arreglo dinámico y realizar análisis sobre este.

Métodos mínimos para implementar:

Se deben implementar las funcionalidades mínimas del Stack y Queue:

Para `MyStack<T>`:

- `push(T x)`: Inserta un elemento en la cima.
- `pop()`: Elimina y retorna el elemento en la cima.
- `peek()`: Retorna el elemento en la cima sin eliminarlo.
- `isEmpty()`: Verifica si la pila está vacía.
- `size()`: Retorna el número de elementos en la pila.
- `delete(n)`: Elimina el primer valor n que encuentra.

Para `MyQueue<T>`:

- `enqueue(T x)`: Inserta un elemento al final.
- `dequeue()`: Elimina y retorna el primer elemento.
- `front()`: Retorna el primer elemento sin eliminarlo.
- `isEmpty()`: Verifica si la cola está vacía.
- `size()`: Retorna el número de elementos en la cola.
- `delete(n)`: Elimina el primer valor n que encuentra.

3. Análisis de Complejidad

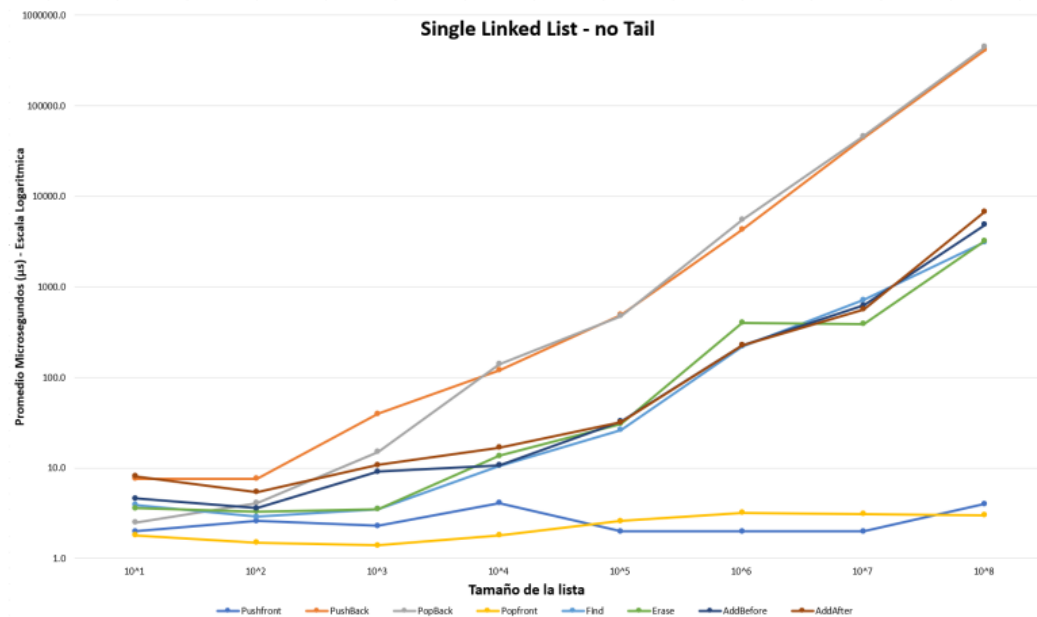
- Formular hipótesis sobre la eficiencia de cada método en las distintas implementaciones.
- Medir el tiempo de ejecución con distintos tamaños de entrada (ej. 10, 100, 10^4 , 10^6 , 10^8). Se recomiendan estas entradas sean aleatorias.
- Comparar los resultados obtenidos con la complejidad teórica en notación Big-O.
- El tiempo de ejecución debe medirse en milisegundos o nanosegundos, se debe justificar la elección de cual uso (TIP: la precisión es primordial)

El análisis debe estar separado en 3 partes:

- 1. Comparativa de cada método de la Estructura **List**, según implementación, comparar todos los métodos al usar con cola, sin cola, etc, con las mencionadas en el apartado 1. Ejemplo de como se debe realizar el análisis:

Single Linked List – No Tail:

Tamaño de lista n	Pushfront	PushBack	PopBack	Popfront	Find	Erase	AddBefore	AddAfter
10^1	6.6	8.8	2.5	2.5	5.3	4.3	6.6	5.8
10^2	12.3	8.9	3.5	2.1	3.3	3.4	4.6	5.1
10^3	2.4	3.4	17.7	2.0	10.4	4.9	4.3	5.3
10^4	3.0	4.4	175.9	2.0	8.9	10.6	10.8	5.9
10^5	3.4	3.7	452.1	3.0	35.3	27.5	16.1	4.2
10^6	2.9	3.3	4895.0	2.0	191.0	306.4	125.8	3.4
10^7	4.1	3.5	44294.3	5.0	512.9	455.9	373.9	5.5
10^8	3.1	4.0	472997.6	3.0	3879.1	2164.8	3501.4	10.5



- 2. Mismo análisis que el anterior pero ahora para las implementaciones de MyStack y MyQueue
- 3. Comparativa de métodos equivalentes, es decir Comparativa List / (Stack, Queue), por ejemplo PushFront en List se deberá comparar con Push de MyStack y pushback se deberá comparar con enqueue de MyQueue, y así con todos los métodos equivalentes. Se debe elegir solo una de las 4 posibles implementaciones de List para realizar la comparativa por método (se puede elegir una implementación diferente para cada método) pero se debe explicar porque para ese método eligió X implementación de List, se busca la opción mas óptima para compararla con la implementación de Circular Arrays en MyStack y MyQueue.

4. Visualización y Graficación de Resultados

- a. Realizar pruebas experimentales para las operaciones implementadas.
- b. Graficar el tiempo de ejecución vs. tamaño de entrada (ver foto ejemplo en apartado 3.) diferenciando implementaciones.
- c. Se recomienda el uso de [Matplotlib](#), u otra librería que permita graficar resultados, sin embargo, el estudiante puede usar cualquier otra herramienta que permita visualizar los resultados. Para el ejemplo de la foto se usó excel para no afectar la medición de los tiempos.
- d. Los tiempos de ejecución se pueden graficar en cualquier medida de tiempo siempre y cuando refleje de forma sencilla e intuitiva la diferencia de tiempos de ejecución entre métodos, ya que por ejemplo si se mide en nanosegundos el código, puede que den números muy grandes en algunos casos lo cual hará perder la percepción de la comparativa, por tanto se recomienda cambiar la escala al de tiempo al graficar por una promedio que permita ver de forma clara la diferencia de tiempos, números muy pequeños tampoco permiten evidenciar diferencias, en la imagen se usó microsegundos para el análisis.

5. Conclusiones

- Detallar bajo cuáles condiciones es mejor usar cada estructura, teniendo en cuenta las diferentes implementaciones de los métodos.
- Reflexionar sobre escenarios de uso de pilas y colas en aplicaciones reales.
- Analizar ventajas y desventajas de arreglos dinámicos vs. listas enlazadas en Java.

ENTREGABLES

1. **Código Fuente:** Repositorio en GitHub con el código, no se aceptaran archivos .zip, todo tiene que estar en un repositorio de GitHub y el link de acceso a este debe estar en el informe de PDF:
2. **Informe en PDF:**
 - Objetivo del taller.
 - Explicación de la implementación.
 - Análisis de complejidad (teórico vs. empírico).
 - Gráficos de resultados (puede ir junto al análisis de complejidad).
 - Conclusiones.

CRITERIOS DE EVALUACIÓN

- Correctitud y completitud de las implementaciones en Java.
- Calidad del análisis de complejidad.
- Claridad y orden en el informe.
- Calidad de los gráficos y coherencia en los resultados obtenidos
- Profundidad de las conclusiones y reflexión sobre los resultados.

REGLAS DE ENTREGA

- **Trabajo individual**
- Entrega por Moodle en 1 archivo:
 - Informe en PDF: `Stack-Queue-Java-ED-# Estudiante.pdf`.
- No se aceptarán entregas que usen librerías para la implementación de las estructuras, si se aceptan librerías para la medición del tiempo y/o graficación de datos.
- Se recomienda tener cuidado al graficar los resultados directamente en el código, ya que esto podría afectar los tiempos de ejecución, la medición del tiempo y la graficación (en caso de hacerla directamente en el código) deben estar separadas correctamente, si se evidencia que la graficación de resultados esta dentro de la medición del tiempo, afectará significativamente la nota ya que todos los resultados estarán sesgados por estas operaciones de graficación adicionales.
- Se recomienda revisar el archivo de Main.java que se encuentra en micampus en el enlace para entregar esta tarea ya que este tiene un borrador o guía sobre como medir los tiempos de ejecución.